

YZ 203 · VISUAL PROGRAMMING

Week 3

Timer & Animation Basics: making things move

Erciyes University

Java Swing & Event-Driven Programming

Murat AKPULAT, PhD

What's on today

One new class, `javax.swing.Timer`, and everything it unlocks: moving things, speed control, and the first version of the Falling Egg game.

01

Timer basics

Events from a clock, delay, start / stop

02

The game loop

Update, render, wait, and why `sleep()` fails

03

OOP behind Timer

Composition, interface reuse, observers, encapsulation

04

Moving things

null layout, bounce, fall, random position

05

Egg class

Our first own class with state and behavior

06

Falling Egg v0

Start / Pause, speed slider, missed counter

Quick Recap: Where We Left Off

Week 1

JFrame, JPanel, JLabel, JButton, text fields, layouts, and the idea of an event.

Week 2

Nine more components and four listener interfaces: Action, Item, ListSelection, Change.

The pattern

Every event so far was caused by the user: a click, a drag, a selection.

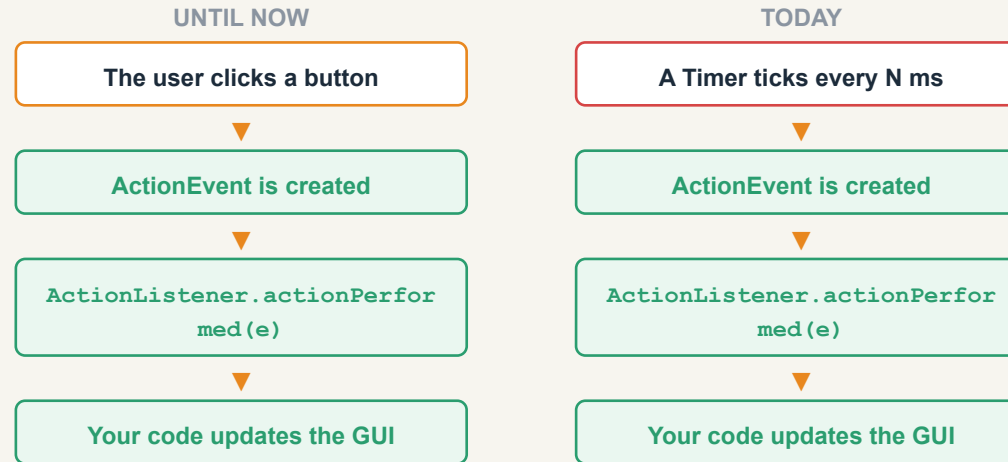
Today

Events caused by the clock. No click needed, the program animates itself.

The listener code you already know stays the same. Only the thing that fires the event changes.

Who Fires the Event When Nobody Clicks?

Same chain as Week 1. Only the first box is new.



The green boxes are identical, which is why Timer feels familiar: it is an event source, like a button, but driven by time.

Anatomy of a Timer

```
Timer timer = new Timer(16, e -> update());
```

16

The delay in milliseconds between two ticks. 16 ms is about 60 ticks per second.

e -> update()

An ActionListener written as a lambda. It runs once per tick.

timer.start()

Creating a Timer does nothing. It only ticks after start() is called.

THE METHODS YOU WILL USE TODAY

`start()``stop()``restart()``isRunning()``setDelay(int)``setRepeats(boolean)`

Your First Timer: a Ticking Label

Scenario: a label that counts seconds by itself. No button, no click.

```
import javax.swing.*;

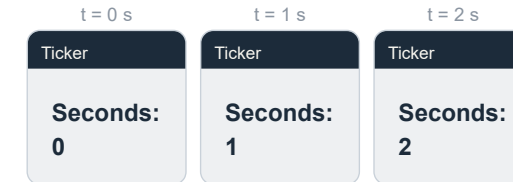
public class TickDemo {
    static int seconds = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Ticker");
        JLabel label = new JLabel("Seconds: 0", SwingConstants.CENTER);

        Timer timer = new Timer(1000, e -> {
            seconds++;
            label.setText("Seconds: " + seconds);
        });
        timer.start();

        f.add(label);
        f.setSize(250, 120);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

- 1. Event source** — the Timer fires an ActionEvent every 1000 ms; the lambda is the listener.
- 2. Interface** — the lambda implements ActionListener, exactly like the JButton code from Week 1.
- 3. Why static?** — a lambda cannot change a normal local variable, so the counter lives in a static field.

Delay, Ticks per Second and How It Feels

The delay is the only knob. Smaller delay means more ticks per second.

delay (ms)	ticks per second	feels like
1000	1	a clock
500	2	a blinking cursor
100	10	choppy but visible motion
33	about 30	film-like
16	about 60	smooth, game quality

Formula: ticks per second = 1000 / delay. Swing timers are not perfectly precise; the delay is a target, not a promise. Our games only need "smooth enough".

The Game Loop: Update, Render, Wait

Every animation, from a bouncing box to a full game, repeats the same three steps.



...then back to step 1. The Timer runs the loop for you, you only write steps 1 and 2 inside one `actionPerformed`.

Why Not Just Use a Loop and sleep()?

Wrong: the window freezes

```
while (true) {  
    x += 3;  
    box.setLocation(x, 50);  
    Thread.sleep(16);  
}
```

- The loop never returns, so Swing's event thread is stuck inside it.
- Nothing repaints, buttons stop responding, the window shows nothing.

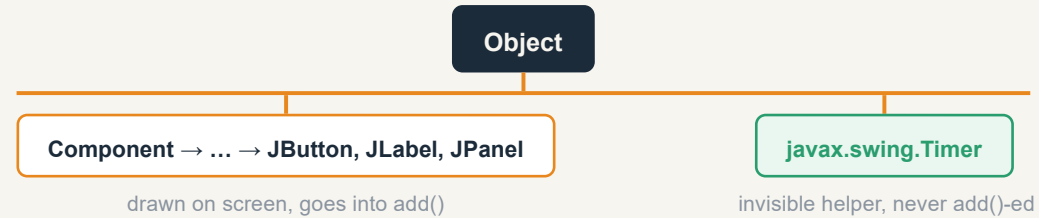
Right: a Timer returns at once

```
new Timer(16, e -> {  
    x += 3;  
    box.setLocation(x, 50);  
}).start();
```

- start() returns immediately; each tick runs briefly and finishes.
- Between ticks the event thread repaints and handles clicks.

OOP 1: Timer Is Not a Component

Look for Timer in the class tree from Week 1. It is not there.

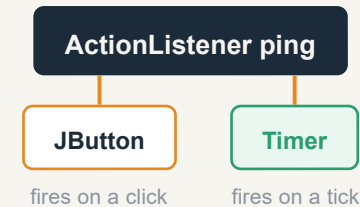


Same idea as ButtonGroup in Week 2: **composition**. A Timer does not draw anything; it holds a list of listeners and calls them. Never write `panel.add(timer)`.

OOP 2: One Listener, Two Different Sources

A listener does not know, or care, who fires it. That is the point of an interface.

```
ActionListener ping =  
    e -> label.setText("Ping!");  
  
button.addActionListener(ping);  
timer.addActionListener(ping);
```



Polymorphism again: both sources accept any ActionListener and only ever call actionPerformed().

OOP 3: Many Listeners, One Timer

`addActionListener()` adds, it does not replace. This is the Observer pattern: one subject, many observers.

FRAGMENT

```
Timer timer = new Timer(1000, e -> seconds++);
timer.addActionListener(
    e -> label.setText("" + seconds));
timer.addActionListener(
    e -> log.append("tick\n"));
timer.start();
```

Timer (1000 ms)

1. seconds++

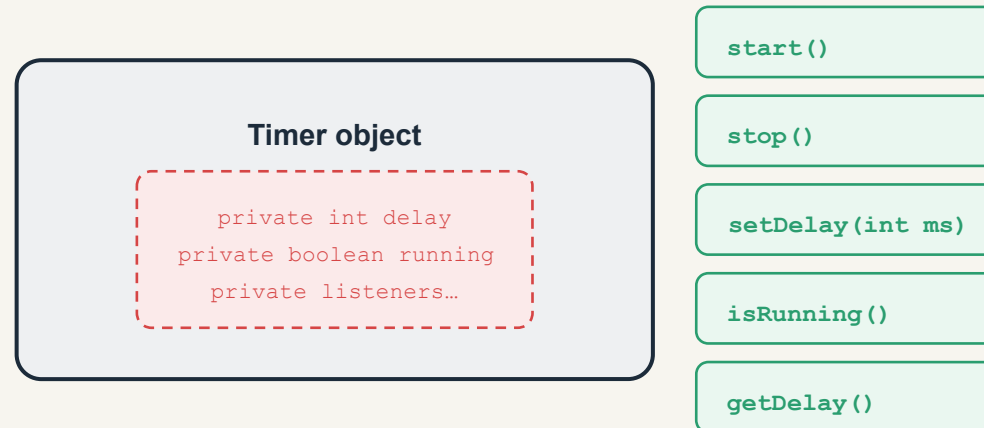
2. update the label

3. append to the log

On every tick the listeners run in the order they were added. Remember this for the game: score, speed and sound can each get their own listener.

OOP 4: Encapsulation, Private State, Public Doors

A Timer hides how it keeps time. You only get a few safe methods.



You can ask `isRunning()`, but you cannot flip a running flag yourself. Same shape as `getText()` on a `JTextField`.

Controlling a Timer with Buttons

Scenario: a counter the user can start and stop. The Timer is a normal object we hold a reference to.

```
import javax.swing.*.*;

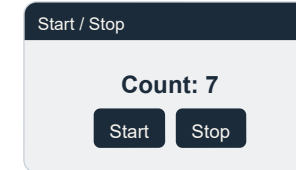
public class StartStopDemo {
    static int count = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Start / Stop");
        JPanel p = new JPanel();
        JLabel label = new JLabel("Count: 0");
        JButton start = new JButton("Start");
        JButton stop = new JButton("Stop");

        Timer timer = new Timer(500,
            e -> label.setText("Count: " + (++count)));
        start.addActionListener(e -> timer.start());
        stop.addActionListener(e -> timer.stop());

        p.add(label); p.add(start); p.add(stop);
        f.add(p);
        f.setSize(300, 120);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT

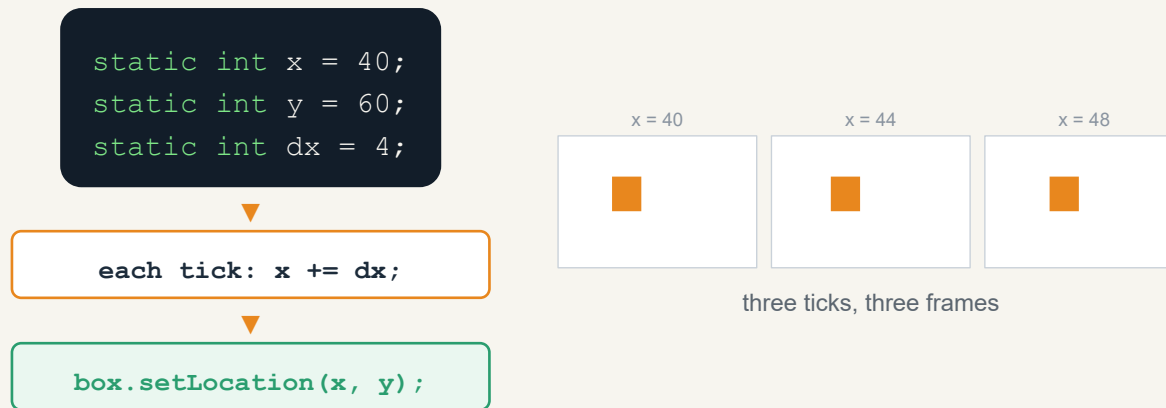


STEP BY STEP — WHERE OOP SHOWS UP

- 1. Reference** — we store the Timer in a variable so other listeners can reach it.
- 2. Events from two sources** — two JButtons and one Timer, all feeding ActionListeners.
- 3. Encapsulation** — start() and stop() are the only doors into the Timer.

Animation Is Just Numbers Changing

The egg does not "move". Its numbers change, and we tell the component where it is now.



State (the numbers) is separate from view (the component). Keep that separation; the Egg class will formalize it.

Recap: Positioning with null Layout

Week 1 promised we would need this for games. A layout manager would fight us by moving the egg back.

```
JPanel panel = new JPanel(null);  
  
egg.setBounds(100, 0, 30, 40);  
egg.setLocation(120, 25);  
egg.setSize(30, 40);
```

method	what it sets
setBounds(x, y, w, h)	position and size
setLocation(x, y)	position only
setSize(w, h)	size only
getX() / getY()	read the position back

Reminder: normal forms and dialogs still use layout managers. null layout is the exception for moving things.

Moving a Component: Box Slides Right

Scenario: an orange box that slides across the window, 3 pixels per tick.

```
import javax.swing.*;
import java.awt.Color;

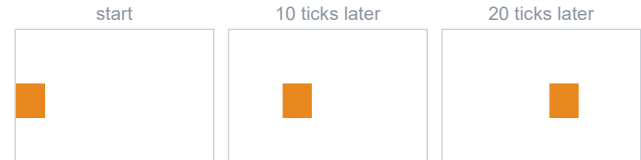
public class MoveRight {
    static int x = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Move Right");
        JPanel panel = new JPanel(null);
        JLabel box = new JLabel();
        box.setOpaque(true);
        box.setBackground(Color.ORANGE);
        box.setBounds(x, 50, 40, 40);
        panel.add(box);

        new Timer(16, e -> {
            x += 3;
            box.setLocation(x, 50);
        }).start();

        f.add(panel);
        f.setSize(420, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

- 1. Inheritance** — JLabel gets `setOpaque()` and `setBackground()` from JComponent, so a plain label can act as a colored box.
- 2. Inherited position** — `setLocation()` comes from Component, the root of the tree.
- 3. State** — `x` lives outside the component; each tick changes it, then we pass it on.

Bouncing Off the Edges

Scenario: the box reverses direction whenever it touches the left or right wall.

```
import javax.swing.*;
import java.awt.Color;

public class BounceDemo {
    static int x = 0, dx = 4;

    public static void main(String[] args) {
        JFrame f = new JFrame("Bounce");
        JPanel panel = new JPanel(null);
        JLabel box = new JLabel();
        box.setOpaque(true);
        box.setBackground(Color.ORANGE);
        box.setBounds(0, 50, 40, 40);
        panel.add(box);

        new Timer(16, e -> {
            x += dx;
            if (x < 0 || x > panel.getWidth() - 40) dx = -dx;
            box.setLocation(x, 50);
        }).start();

        f.add(panel);
        f.setSize(420, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

- 1. One variable, two directions** — dx is +4 going right, -4 going left. Negating it flips the motion.
- 2. Use $<$ and $>$, never $==$** — with a step of 4 the box may jump over the exact edge value.
- 3. Ask the container** — `panel.getWidth()` adapts if the window is resized.

An Egg Falls and Starts Over

Scenario: the egg drops from the top; when it passes the bottom it returns to the top.

```
import javax.swing.*;
import java.awt.Color;

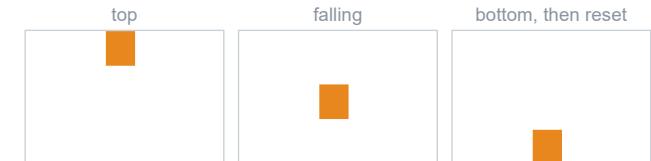
public class FallDemo {
    static int y = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Fall");
        JPanel panel = new JPanel(null);
        JLabel egg = new JLabel();
        egg.setOpaque(true);
        egg.setBackground(Color.ORANGE);
        egg.setBounds(100, 0, 30, 40);
        panel.add(egg);

        new Timer(16, e -> {
            y += 4;
            if (y > panel.getHeight()) y = 0;
            egg.setLocation(100, y);
        }).start();

        f.add(panel);
        f.setSize(260, 300);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. **Same loop, other axis** — y grows downward in Swing: 0 is the top edge.
2. **Reset rule** — the if-statement is the game logic; everything else is plumbing.
3. **Next** — we want the egg to restart at a random x. That needs Random.

Random Positions with java.util.Random

Random is a plain Java class, nothing to do with Swing. Create it once, then ask it for numbers.

```
static Random rnd = new Random();

int a = rnd.nextInt(5);
int x = rnd.nextInt(panel.getWidth() - 30);
```

call	result
<code>rnd.nextInt(5)</code>	0, 1, 2, 3 or 4
<code>rnd.nextInt(100)</code>	0 ... 99
<code>nextInt(width - 30)</code>	x that keeps a 30 px egg inside

Upper bound is exclusive. Create the Random once (a static field), not inside the tick, otherwise you rebuild it 60 times a second.

Falling Egg with a Random Column

Scenario: every new egg appears at a different x, like the real game.

```
import javax.swing.*;
import java.awt.Color;
import java.util.Random;

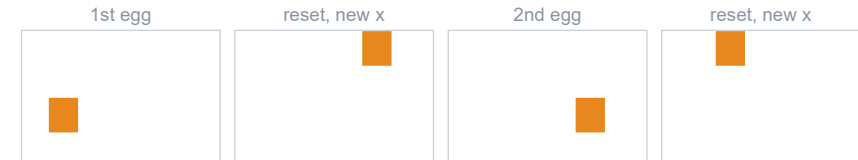
public class RandomFall {
    static int y = 0;

    public static void main(String[] args) {
        Random rnd = new Random();
        JFrame f = new JFrame("Random Fall");
        JPanel panel = new JPanel(null);
        JLabel egg = new JLabel();
        egg.setOpaque(true);
        egg.setBackground(Color.ORANGE);
        panel.add(egg);

        new Timer(16, e -> {
            y += 4;
            if (y > panel.getHeight()) {
                y = 0;
                egg.setLocation(rnd.nextInt(panel.getWidth() - 30), 0);
            }
            egg.setBounds(egg.getX(), y, 30, 40);
        }).start();

        f.add(panel);
        f.setSize(260, 300);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

- 1. Composition** — Random is just an object we call; no inheritance involved.
- 2. Reset in one place** — new x and y = 0 happen together inside the if.
- 3. Where the size comes from** — setBounds() each tick gives the egg its 30 x 40 size.

Speed Control: JSlider Meets Timer

Scenario: Week 2 meets Week 3. Dragging the slider changes how fast the box moves.

```
import javax.swing.*;
import java.awt.*;

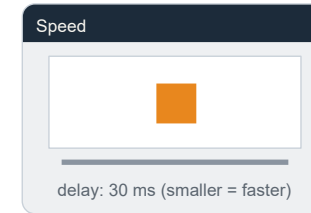
public class SpeedDemo {
    static int x = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Speed");
        JPanel panel = new JPanel(null);
        JLabel box = new JLabel();
        box.setOpaque(true);
        box.setBackground(Color.ORANGE);
        box.setBounds(0, 40, 40, 40);
        panel.add(box);
        JSlider speed = new JSlider(5, 100, 30);

        Timer timer = new Timer(30, e -> {
            x = (x + 4) % panel.getWidth();
            box.setLocation(x, 40);
        });
        speed.addChangeListener(e -> timer.setDelay(speed.getValue()));
        timer.start();

        f.add(panel);
        f.add(speed, BorderLayout.SOUTH);
        f.setSize(420, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

- 1. Four listener types in one app** — `ChangeListener` (slider) controls an `ActionListener` source (timer).
- 2. Live reconfiguration** — `setDelay()` takes effect immediately, no restart needed.
- 3. Modulo** — `x % width` wraps the box back to the left edge.

A Progress Bar That Fills Itself

Scenario: a loading bar for the game, filled by a *Timer* and stopped when full.

```
import javax.swing.*;

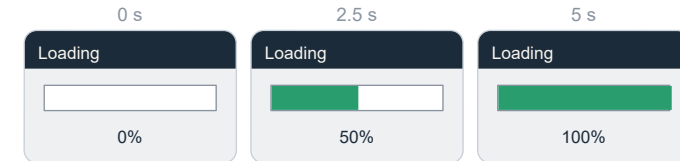
public class AutoProgress {
    static int value = 0;

    public static void main(String[] args) {
        JFrame f = new JFrame("Loading");
        JProgressBar bar = new JProgressBar(0, 100);
        bar.setStringPainted(true);

        Timer timer = new Timer(50, null);
        timer.addActionListener(e -> {
            value++;
            bar.setValue(value);
            if (value >= 100) timer.stop();
        });
        timer.start();

        f.add(bar);
        f.setSize(300, 90);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

- 1. Why new `Timer(50, null)`?** — a lambda cannot mention timer inside its own declaration, so we add the listener afterwards.
- 2. A timer can stop itself** — `timer.stop()` is called from inside its own listener.
- 3. Composition** — Timer drives, JProgressBar displays; neither knows the other exists.

Countdown: Stop When the Condition Is Met

Scenario: 5, 4, 3, 2, 1, GO! Exactly the ready-steady-go screen the game will need.

```
import javax.swing.*;

public class Countdown {
    static int left = 5;

    public static void main(String[] args) {
        JFrame f = new JFrame("Countdown");
        JLabel label = new JLabel("5", SwingConstants.CENTER);
        label.setFont(label.getFont().deriveFont(48f));

        Timer timer = new Timer(1000, null);
        timer.addActionListener(e -> {
            left--;
            label.setText(left > 0 ? "" + left : "GO!");
            if (left == 0) timer.stop();
        });
        timer.start();

        f.add(label);
        f.setSize(220, 140);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

- 1. State machine** — the value of `left` decides what the label shows and when the timer ends.
- 2. Ternary expression** — condition `? a : b` picks the text in one line.
- 3. Cleanup** — always stop a timer that has finished its job.

One-Shot Timer: Show a Message, Then Hide It

Scenario: "Saved!" appears on a click and disappears two seconds later, once.

```
import javax.swing.*.*;

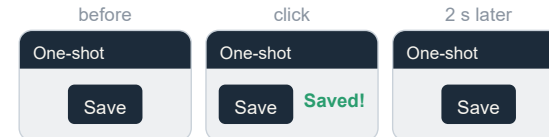
public class OneShot {
    public static void main(String[] args) {
        JFrame f = new JFrame("One-shot");
        JPanel p = new JPanel();
        JLabel status = new JLabel(" ");
        JButton save = new JButton("Save");

        Timer hide = new Timer(2000, e -> status.setText(" "));
        hide.setRepeats(false);

        save.addActionListener(e -> {
            status.setText("Saved!");
            hide.restart();
        });

        p.add(save); p.add(status);
        f.add(p);
        f.setSize(260, 100);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

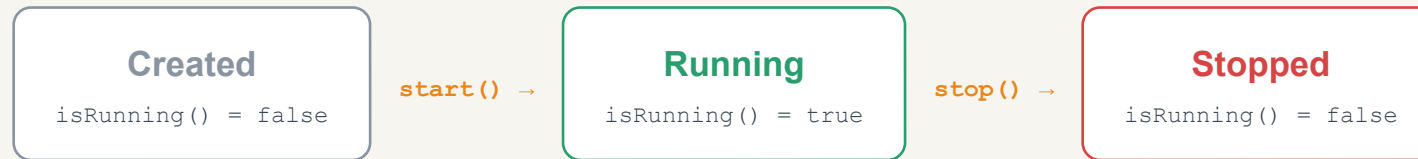
OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

- 1. setRepeats(false)** — the timer fires once and stops itself.
- 2. restart()** — clicking Save again cancels the pending hide and starts the 2 s over.
- 3. Three objects, one flow** — JButton, JLabel and Timer cooperate through listeners only.

A Timer's Life: Created, Running, Stopped



`start()` works again from **Stopped**. `restart()` stops and starts in one call. A timer may also call `stop()` on itself from inside its own listener, as the progress bar and countdown did.

Our First Own Class: Egg

So far x, y and dx floated around as static variables. An egg should own its state and know how to move.

Egg	
- x : int	
- y : int	
- speed : int	
- view : JLabel	
+ Egg(x, speed)	
+ fall()	
+ getView() : JLabel	

```
// needs: javax.swing.*, java.awt.Color
class Egg {
    private int x, y;
    private final int speed;
    private final JLabel view = new JLabel();

    Egg(int x, int speed) {
        this.x = x;
        this.speed = speed;
        view.setOpaque(true);
        view.setBackground(Color.ORANGE);
        view.setBounds(x, 0, 30, 40);
    }

    void fall() {
        y += speed;
        view.setLocation(x, y);
    }

    JLabel getView() { return view; }
}
```

- **private** fields are hidden (encapsulation). + **public** methods are the doors. Week 10 returns to this properly; today it is just a good habit.

Using the Egg Class with a Timer

Before: loose variables

```
static int x, y, speed;  
// one egg = 3 variables  
// two eggs = 6 variables  
// ten eggs = ???
```

State and behavior live in different places.

After: an Egg object

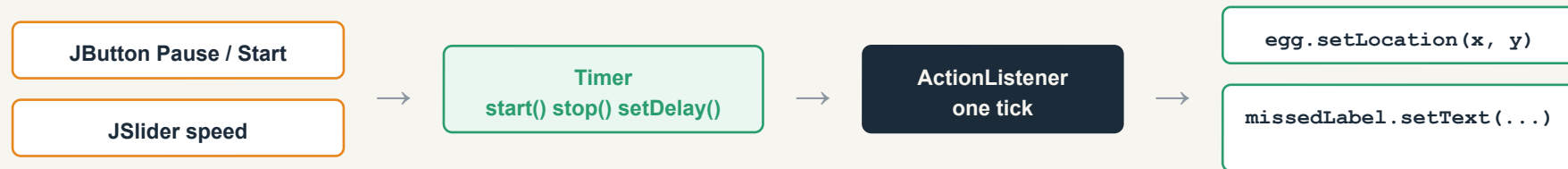
```
Egg egg = new Egg(120, 4);  
panel.add(egg.getView());  
  
new Timer(16, e -> egg.fall()).start();
```

The Timer asks the egg to fall. The egg knows how.

Later weeks put many Egg objects into an ArrayList and call fall() on each one. No new variables needed.

Falling Egg v0: How the Pieces Connect

Before writing the code, the data flow. Two inputs control one Timer; one listener moves the egg and counts misses.



Components used: JPanel(null), JLabel x2, JButton, JSlider, Timer, Random. Nothing new except how they are wired.

Falling Egg v0, Part 1: Build the Screen

Everything from Weeks 1 to 3 on one screen. The full file is in the lab notes.

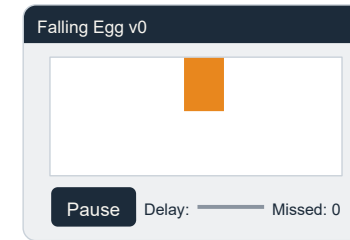
```
import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class FallingEgg {
    static int y = 0, missed = 0;
    static final Random rnd = new Random();

    public static void main(String[] args) {
        JFrame f = new JFrame("Falling Egg v0");
        JPanel field = new JPanel(null);
        JLabel egg = new JLabel();
        egg.setOpaque(true);
        egg.setBackground(Color.ORANGE);
        egg.setBounds(100, 0, 30, 40);
        field.add(egg);

        JPanel bar = new JPanel();
        JButton toggle = new JButton("Pause");
        JSlider speed = new JSlider(5, 40, 16);
        JLabel missedLabel = new JLabel("Missed: 0");
        bar.add(toggle); bar.add(new JLabel("Delay:"));
        bar.add(speed); bar.add(missedLabel);
        // ... continued on the next slide
    }
}
```

OUTPUT



WHAT TO NOTICE

- 1. Two panels** — a null-layout field for the egg, a normal FlowLayout bar for the controls.
- 2. Composition** — the frame holds the field, the field holds the egg: the Week 1 hierarchy again.
- 3. State** — `y` and `missed` are static fields so the lambda in Part 2 can change them.

Falling Egg v0, Part 2: Wire the Timer

The tick listener moves and counts; the button and slider control the Timer itself.

CONTINUATION OF PART 1

```
Timer timer = new Timer(16, e -> {
    y += 4;
    if (y > field.getHeight()) {
        missed++;
        missedLabel.setText("Missed: " + missed);
        y = 0;
        egg.setLocation(rnd.nextInt(field.getWidth() - 30), 0);
    }
    egg.setLocation(egg.getX(), y);
});
toggle.addActionListener(e -> {
    if (timer.isRunning()) { timer.stop(); toggle.setText("Start"); }
    else { timer.start(); toggle.setText("Pause"); }
});
speed.addChangeListener(e -> timer.setDelay(speed.getValue()));

f.add(field);
f.add(bar, BorderLayout.SOUTH);
f.setSize(420, 400);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
f.setVisible(true);
timer.start();
}
```

OUTPUT



WHAT TO NOTICE

- 1. Three interfaces, one Timer** — ActionListener ticks, ActionListener button, ChangeListener slider.
- 2. Encapsulation** — `isRunning()` decides what the button does; we never touch a flag.
- 3. This is a game** — it has state, a loop, input and a score. It is just not interactive with the egg yet.

Two Classes Named Timer

The most common import mistake of the week. Check the import line.

	<code>javax.swing.Timer</code>	<code>java.util.Timer</code>
Runs your code on	Swing's event thread	its own background thread
Safe to touch components?	Yes	No, needs <code>SwingUtilities.invokeLater</code>
Constructor	<code>new Timer(delay, listener)</code>	<code>new Timer() + schedule(task, ...)</code>
Use it for	GUI animation, our games	background jobs without a GUI

With `import javax.swing.*`; plain `Timer` means the right one. If you also import `java.util.*`, write `javax.swing.Timer` explicitly.

Reference: The Timer API Used This Week

member	purpose
<code>new Timer(int delay, ActionListener l)</code>	create a repeating timer, not yet running
<code>start() / stop()</code>	begin / end ticking
<code>restart()</code>	stop, reset the delay counter and start again
<code>isRunning()</code>	is it currently ticking?
<code>setDelay(int ms) / getDelay()</code>	change / read the time between ticks
<code>setRepeats(boolean)</code>	false = fire once, then stop
<code>addActionListener(l)</code>	add one more listener (does not replace)

Common Mistakes I: Nothing Moves

Forgot `timer.start()`

Symptom: the window opens, the egg never moves, no error.

Fix: a `Timer` only ticks after `start()` is called.

Used `java.util.Timer`

Symptom: compile error, "constructor `Timer` cannot be applied".

Fix: import `javax.swing.Timer`, or use only import `javax.swing.*`;

Forgot `setLayout(null)` on the panel

Symptom: `setLocation` / `setBounds` have no effect, the component stays centered or fills the panel.

Fix: create the panel as `new JPanel(null)` when you position by hand.

Common Mistakes II: Compile Errors Around Lambdas

Changing a local variable inside the lambda

Symptom: "local variables referenced from a lambda must be final or effectively final".

Fix: move the counter to a static field (or later, a field of an object).

Using timer inside its own declaration

Symptom: "variable timer might not have been initialized".

Fix: create `Timer(50, null)` first, then call `timer.addActionListener(...)`.

Edge check with ==

Symptom: the box passes the wall and never bounces back.

Fix: with a step of 4 the position may skip the exact value. Use `<` or `>`.

Common Mistakes III: Behavior Problems

Thread.sleep() inside a listener or loop

Symptom: the window freezes or never appears.

Fix: never block the event thread; let the Timer do the waiting.

A tiny delay such as 1 ms

Symptom: high CPU, no visible gain; ticks are not that precise.

Fix: use 10 to 20 ms for smooth animation.

Never stopping a finished timer

Symptom: score or progress keeps updating after "game over" or 100%.

Fix: call timer.stop() when the end condition is reached.

Lab Goals I: Warm-up (Easy)

- 1. Ticker: a label that counts seconds by itself
- 2. Digital clock: show the real time, updated every second
- 3. Start / Stop: control the counter with two buttons
- 4. Auto progress: a JProgressBar that fills and stops at 100%
- 5. Move right: slide a box across the window

Lab Goals II: Towards the Game (Medium to Hard)

- 6. Bounce: reverse direction at both walls
- 7. Random fall: new random column and a missed counter
- 8. Speed slider: change the delay while it runs
- 9. Egg class: move the state and fall() into an object
- 10. Falling Egg v0: Start / Pause, slider and missed label
- 11. Challenge: two eggs falling at different speeds

NEXT WEEK

Week 4: Graphics & paintComponent

Our eggs are colored labels. Next week we draw them for real: a JPanel subclass, paintComponent(), Graphics, and the egg and the basket as shapes. Inheritance and overriding finally get a starring role.

Questions?